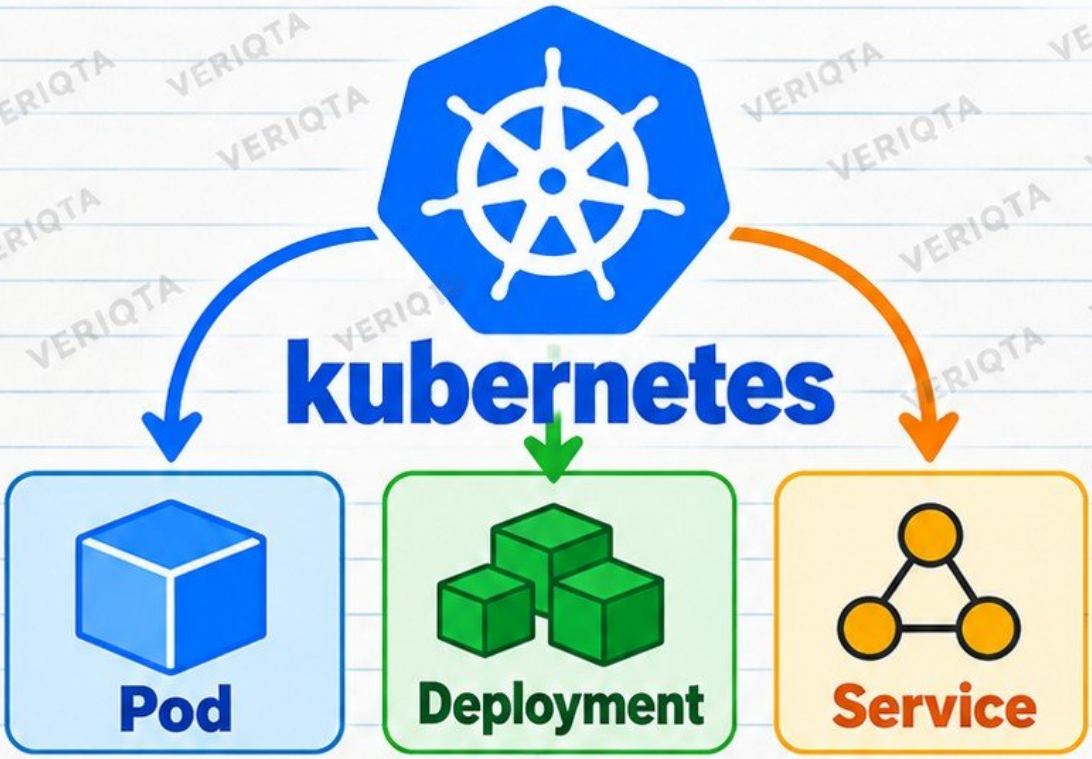
VERIQTA

Kubernetes for Beginners:

Your First Pod, Service, and Deployment



- Step-by-step, beginner-friendly guide
- Real commands and practical examples
- Troubleshooting and common failures
- Learn by building a real application

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

1. What Is Kubernetes, and Why Do Engineers Need It?

What Is Kubernetes?

Kubernetes is an open source system for running, managing, and scaling containerized applications.
It helps engineers maintain a desired application state, even when things fail.

kubernetes

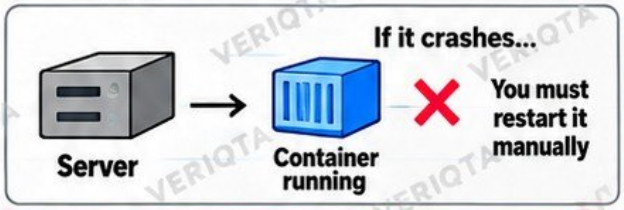
Key Idea

You tell Kubernetes what you want (e.g. 3 replicas), and it works to keep that state, even if containers crash or servers fail.

The Problem with Managing Containers Manually

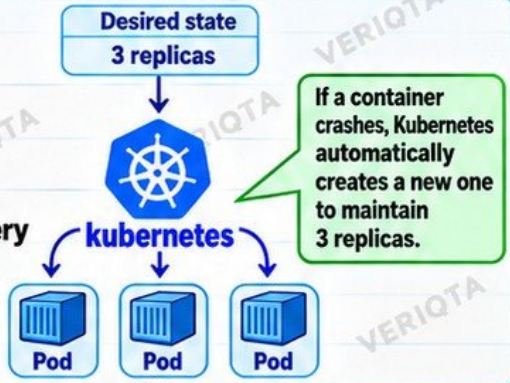
If you run containers manually (for example with Docker), you must:

- Start containers yourself
- Restart them when they crash
- Replace them when a server fails
- Manage load balancing yourself
- Scale them manually when traffic increases



Kubernetes Solves This Problem

- Automatically keeps your application running
- Replaces failed containers
- Runs containers across multiple servers (nodes)
- Provides networking, load balancing, and service discovery
- Scales your application up or down
- Helps with updates, rollbacks, and self-healing



What Happens When a Container Crashes or a Server Becomes Unavailable?

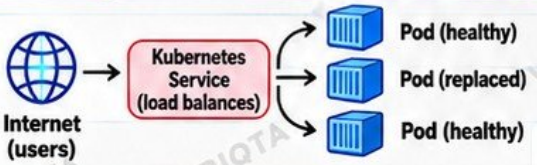
- Manually: the application goes down until you notice and restart it.
- With Kubernetes: it detects the failure and replaces the container, or moves it to another healthy node.

Kubernetes vs Docker

Docker	Kubernetes
Builds container images	Runs and manages containers at scale
Runs containers on a single machine	Keeps desired state
Focuses on container runtime	Handles networking, scaling, and self-healing
	Works across multiple servers (cluster)

Real-World Example: Keeping a Web Application Running

You have an online shop running in containers.
If one container crashes or a server goes down, Kubernetes automatically replaces the container and keeps the application available for users without manual intervention.



VERIQTA

2. Inside a Kubernetes Cluster: Who Does What?

Kubernetes Cluster

- A cluster is a group of servers that run Kubernetes.
- It has a control plane and worker nodes.
- The cluster works together to run and manage your applications.

Control Plane

- Manages the cluster.
- Makes decisions.
- Keeps the desired state of your applications.
- Includes the API server, scheduler, and controllers.

Worker Nodes

- Run your application containers (Pods).
- Contain the kubelet and container runtime.
- Can be one or many nodes.
- Kubernetes automatically places Pods on healthy nodes.

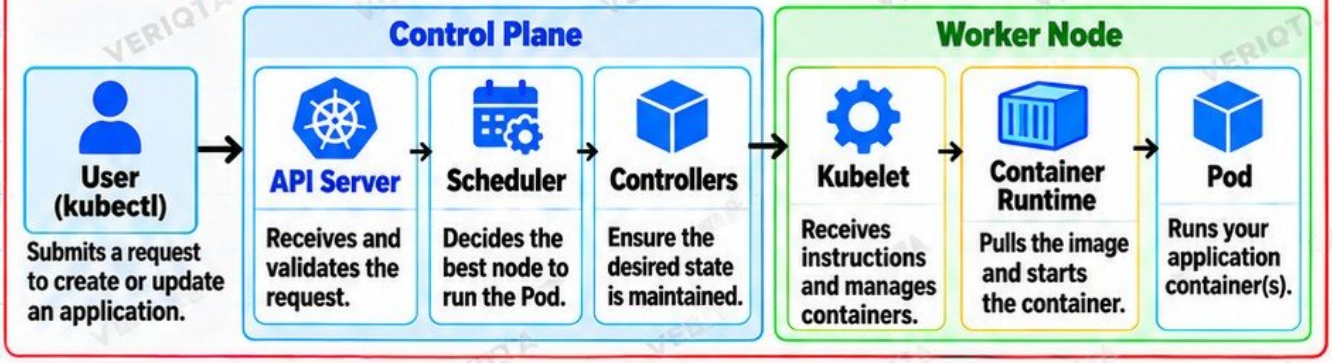
Key Kubernetes Components

API Server	Receives requests (e.g. from kubectl), validates them, and communicates with the cluster.
Scheduler	Decides which worker node should run a new Pod based on available resources and rules.
Controllers	Watch the cluster state and take action to match the desired state (e.g. create, replace, or scale Pods).
Kubelet	Runs on each worker node. It receives instructions from the control plane and makes sure containers are running.
Container Runtime	Runs the actual containers (e.g. containerd, Docker, CRI-O).
Pod	The smallest deployable unit in Kubernetes. It contains one or more containers.

How Kubernetes Decides Where to Run a Pod

1. You submit a request (e.g. a Deployment).
2. The API server saves the request and validates it.
3. The scheduler checks available nodes and selects the best node.
4. The kubelet on that node receives the instruction.
5. The container runtime pulls the image and starts the container.
6. The Pod runs on the selected worker node.

How It All Works Together



Real-World Example

You deploy a web application using a Deployment. Kubernetes schedules the Pods on available worker nodes. If a node fails, Kubernetes can move or recreate the Pods on another healthy node, keeping your application running.

Beginner Insight

You do not manually start containers on nodes. Kubernetes manages workloads through its API. You tell Kubernetes what you want (desired state), and it takes care of the rest.

3. Build Your First Kubernetes Practice Environment

1. What We Will Do

1. Install and understand kubectl and kind.
2. Create a local Kubernetes cluster.
3. Verify the active context.
4. Inspect the cluster nodes.
5. Learn common failures and fixes.
6. Follow security best practices.

2. Tools We Will Use

kubectl

- Command line tool to communicate with Kubernetes.
- Used to create, view, and manage cluster resources.

kind

- Runs a local Kubernetes cluster using Docker containers.
- Perfect for learning and testing on your laptop.

3. Install kubectl and kind

Install kubectl (Linux example)

```
curl -LO "https://dl.k8s.io/
release/v1.29.0/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

Install kind (Linux example)

```
curl -Lo ./kind https://github.com/kubernetes-
sigs/kind/releases/latest/download/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

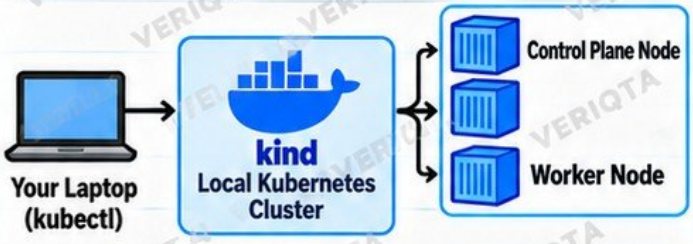
Verify the installations

```
kubectl version --client
kind version
```

You should see version information for both tools.

4. Create a Local Kubernetes Cluster

```
kind create cluster --name veriqta-cluster
```



5. Verify the Active Context

```
kubectl config current-context
```

You should see the name of your kind cluster, for example:

```
veriqta-cluster
```

6. Inspect the Nodes

```
kubectl get nodes
```

Example output:

NAME	STATUS	ROLES	AGE	VERSION
veriqta-cluster-control-plane	Ready	control-plane	2m	v1.29.0
veriqta-cluster-worker	Ready	<none>	2m	v1.29.0

7. Common Failure: kubectl Cannot Connect

Error: The connection to the server localhost:8080 was refused – did you specify the right host or port?

Possible causes:

- You have not created a cluster.
- Your current context is not set to your kind cluster.
- Docker is not running.
- The cluster was deleted or not created successfully.

8. Fix the Connection Issue

1. Check if Docker is running.
2. Verify your current context:
   `kubectl config current-context`
3. If no cluster exists, create it again:
   `kind create cluster --name veriqta-cluster`
4. Check nodes:
   `kubectl get nodes`

9. Security Reminder

- Always confirm your current context before making changes.
- Make sure you are working in the correct cluster (especially when using multiple clusters).
- Avoid running dangerous commands on the wrong cluster.

Run this before making changes:

```
kubectl config current-context
```

It should show the expected cluster name.

4. What Is a Pod? Your First Application Unit

What Is a Pod?

- A Pod is the smallest deployable workload unit in Kubernetes.
- It can contain one or more containers that share the same network and storage.
- Pods are created, managed, and replaced by Kubernetes.

Why Do Engineers Use Pods?

- They provide a reliable way to run containers in a Kubernetes cluster.
- Kubernetes manages Pods so your application keeps running, even if a container or node fails.
- Pods make it easy to scale, update, and recover applications.

Pod vs Container

Container



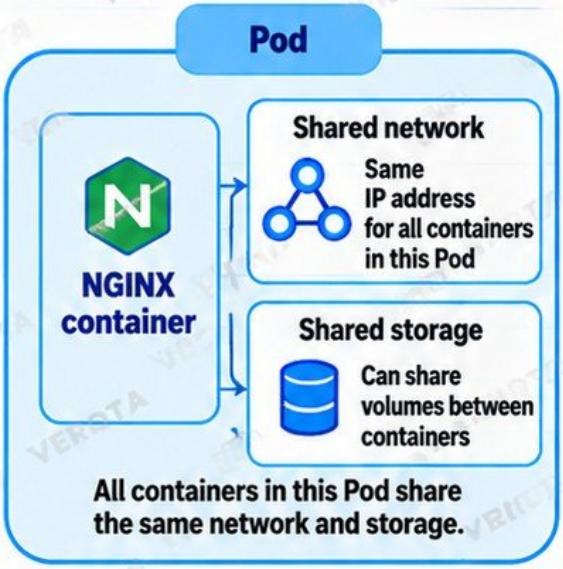
- Runs a single application process.
- Has its own filesystem, but no shared network with other containers.
- Docker creates containers.
- A container can run on your laptop or inside a Pod.

Pod



- A wrapper for one or more containers.
- Containers in a Pod share the same network and storage.
- Created and managed by Kubernetes.
- Pods are scheduled to run on cluster nodes.

Inside a Pod



Why a Pod Is Not the Same as a Container

- A container is just a process with its own filesystem.
- A Pod is a Kubernetes resource that can contain one or more containers.
- Containers in the same Pod share the same IP address, ports, and storage volumes.
- Pods are managed by Kubernetes and can be automatically replaced.
- You can have a Pod with a single container, but a container on its own is not a Pod.

Real-World Example



This Pod runs an NGINX web server inside your Kubernetes cluster. If the container crashes or the node fails, Kubernetes creates a new Pod to keep your application running.

Production Insight



- Pods are replaceable, not permanent servers.
- Kubernetes can delete a Pod at any time and create a new one.
- You should never store important data only inside a Pod.
- Use persistent storage (PersistentVolume) for data that must survive Pod restarts.

5. Create Your First Pod Using kubectl

What We Will Do

1. Create an NGINX Pod using a real container image.
2. Inspect the Pod using kubectl.
3. Understand the output (READY, STATUS, RESTARTS, AGE).
4. Verify that the Pod becomes Ready.
5. Learn a common beginner mistake.

What Is an NGINX Pod?

- We will run an NGINX web server inside a Pod using an official container image.
- The Pod will run in your local Kubernetes cluster (e.g. kind).
- NGINX is a real application that serves a web page.
- This is a simple and safe way to create your first Pod and understand how it works.

1. Create the Pod

```
kubectl run web-pod --image=nginx:stable --port=80
```

- Creates a Pod named web-pod.
- Uses the official NGINX image (nginx:stable).
- Exposes port 80 inside the container.

What Gets Created?



2. Inspect the Pod

```
kubectl get pods
```

```
NAME      READY   STATUS    RESTARTS   AGE
web-pod   1/1     Running   0          24s
```

3. Understand the Output

Column	What It Means
NAME	The name of the Pod.
READY	Number of ready containers / total containers.
STATUS	Current state of the Pod (Running, Pending, etc).
RESTARTS	How many times the container has restarted.
AGE	How long the Pod has been running.

4. Verify the Pod Becomes Ready

- After a few seconds, the STATUS should be Running and READY should be 1/1.
- If it stays in a different state (e.g. Pending or ImagePullBackOff), check the events:

```
kubectl describe pod web-pod
```

- You can also check the logs (for NGINX):

```
kubectl logs web-pod
```

5. Test the Application (Optional)

You can see the NGINX welcome page using port forwarding:

```
kubectl port-forward pod/web-pod 8080:80
```

Then open your browser and go to: `http://localhost:8080`



Beginner Mistake

- Seeing STATUS as Running does not always mean your application is working correctly.
- A Pod can be Running but the application inside can still be misconfigured, unhealthy, or not responding.

Key Takeaway

- You have just created your first Pod.
- You learned how to inspect its status.
- READY, STATUS, RESTARTS, and AGE tell you the health and history of the Pod.
- Always verify that the application is actually working.

6. Inspect a Pod Like a Real DevOps Engineer

1. Describe the Pod

```
kubectl describe pod web-pod
```

Shows detailed information about the Pod, including conditions, container state, events, and more.

```
Name:             web-pod
Namespace:        default
Status:           Running
IP:               10.244.0.12
Nodes:            kind-worker
Containers:
  nginx:
    Image:        nginx:stable
    State:        Running
    Ready:        True
    Restart Count: 0
Events:
  Type    Reason   Age   From      Message
  ----    ------   ----  -------   -----------------------
  Normal  Pulled   30s   kubelet   Successfully pulled image
  Normal  Created  28s   kubelet   Created container
  Normal  Started  28s   kubelet   Started container
```

2. View Pod Logs

```
kubectl logs web-pod
```

Shows the application logs from the container inside the Pod.

```
/docker-entrypoint.sh: /docker-entrypoint.d/
/docker-entrypoint.sh: Looking for shell scripts
/docker-entrypoint.sh: Launching /docker-entrypoint
10.0.0.1 - - [18/Oct/2026:10:05:12 +0000]
"GET / HTTP/1.1" 200 615 "-" "curl/8.4.0"
10.0.0.1 - - [18/Oct/2026:10:06:02 +0000]
"GET /favicon.ico HTTP/1.1" 404 555 "-" "curl/8.4.0"
```

3. Get More Details

```
kubectl get pod web-pod -o wide
```

Shows the Pod with additional details such as node, IP, and container image.

```
NAME      READY  STATUS   RESTARTS  AGE  IP           NODE
web-pod   1/1    Running  0         2m   10.244.0.12  kind-worker
```

4. Understand Pod Conditions

Pod conditions tell you if the Pod is ready to run traffic.

Condition	Meaning
Initialized	All initialization steps are complete.
Ready	The Pod is ready to receive traffic.
ContainersReady	All containers in the Pod are ready.
PodScheduled	The Pod has been assigned to a node.

5. Understand Container State

The container state tells you what is happening inside the container.

State	Meaning
Running	Container is running normally.
Waiting	Container is not running (e.g. pulling image).
Terminated	Container has stopped.

6. Check Events

Events show what has happened to the Pod.

```
kubectl describe pod web-pod | grep--A 10 Events
```

```
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  -------            ----------------------
  Normal  Scheduled  2m    default-scheduler  Successfully assignecd
  Normal  Pulled     2m    kubelet            Successfully pulled image
  Normal  Created    2m    kubelet            Created container
  Normal  Started    2m    kubelet            Started container
```

7. Key Fields to Check

These fields help you quickly understand the Pod.

Field	What to Look For
STATUS	Running, Pending, CrashLoopBackOff, etc.
READY	1/1 means all containers are ready.
RESTARTS	High numbers indicate a problem.
EVENTS	Look for errors or warnings.
LOGS	Check application logs for issues.
POD IP	Confirms the Pod is running and has network.

8. Troubleshooting Example

A Pod exists but the application is not responding.

1. Check Pod status: `kubectl get pods`
2. Describe the Pod and check events: `kubectl describe pod web-pod`
3. Check container logs: `kubectl logs web-pod`
4. Verify the application inside the Pod (if needed): (if needed): `kubectl exec -it web-pod -- curl localhost`
5. Fix the issue based on the evidence (e.g. wrong config, port, or application error).

9. Evidence-First Approach

- Check the Pod status.
- Look at events for clues.
- Check container logs.
- Verify the container is running correctly.
- Only make changes after you understand the evidence.

Common Mistakes

- Assuming Running means the application is working.
- Ignoring events and logs.
- Restarting or deleting the Pod without checking the cause.
- Not checking the correct namespace.

7. Write Your First Pod YAML Manifest

1. Why Use Declarative YAML?

- YAML lets you define the desired state of your application.
- It is version controlled and can be shared.
- You can recreate the same resources anytime.
- It is the standard way to manage resources in real environments.
- Commands are useful for quick testing, but YAML is better for long-term use and consistency.

2. Key Sections in a Pod YAML

Field	What It Means
apiVersion	The Kubernetes API version to use (e.g. v1).
kind	The type of resource (Pod).
metadata	Information about the resource (e.g. name, labels).
spec	The desired configuration for the Pod (e.g. containers, ports, volumes).

3. Pod YAML Example

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
spec:
  containers:
  - name: nginx
    image: nginx:stable
    ports:
    - containerPort: 80
```



4. Understand the YAML Sections

apiVersion: v1
Specifies which Kubernetes API version to use.

kind: Pod
Defines the resource type. Here we are creating a Pod.

metadata:
Contains the name and labels for the Pod.

spec:
Defines the desired state, including containers and their configuration.

5. Apply the Manifest

Save the content to a file named pod.yaml and run:

```
kubectl apply -f pod.yaml
```

- Creates the Pod in your Kubernetes cluster.
- If the Pod already exists, it updates it to match the desired configuration.
- You should see: pod/web-pod created

6. Verify the Pod

Check the Pod status:

```
kubectl get pods
```

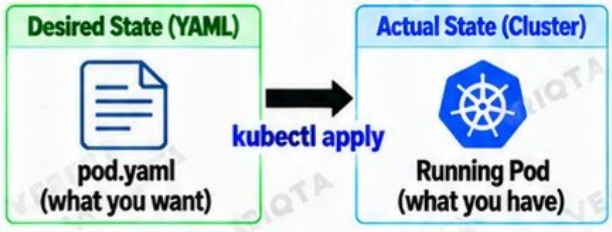
```
NAME      READY   STATUS    RESTARTS   AGE
web-pod   1/1     Running   0          24s
```

Get more details:

```
kubectl describe pod web-pod
```

7. Compare Desired vs Actual State

Your YAML file is the desired state. Use `kubectl` to check the actual state in the cluster.



8. Common Failures and Fixes

Common Issue	How to Fix
Invalid YAML indentation	Make sure indentation is correct (use spaces, not tabs).
Incorrect field name	Check the spelling (e.g. containerPort not containerport).
Image not found	Use a valid image name (e.g. nginx:stable).
Pod stays in ImagePullBackOff	Check the image name and your internet connection.
Pod created but not running	Use `kubectl describe pod web-pod` to check events and logs.

8. Break Your Pod and Learn to Troubleshoot It

1. Create a Pod with an Invalid Image

We will intentionally use a wrong image name to trigger a failure.

```
kubectl run bad-pod \
  --image=nginx:invalid \
  --port=80
```

- This image does not exist.
- The Pod will be created, but the container will fail to start.
- This helps us learn how to troubleshoot.

2. Check the Pod Status

Inspect the Pod using `kubectl get pods`.

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
bad-pod	0/1	ImagePullBackOff	0	1m

- The Pod is created but not ready.
- STATUS shows ImagePullBackOff.
- This means Kubernetes cannot pull the container image from the registry.

3. Investigate Using describe and Events

Use `kubectl describe` to see detailed information and events.

```
kubectl describe pod bad-pod
```

```
Name:         bad-pod
Namespace:    default
Status:       Pending
Containers:
  bad-pod:
    Image:          nginx:invalid
    State:          Waiting
      Reason:       ImagePullBackOff
    Ready:          False
Events:
  Type     Reason     Age   From               Message
  ----     ------     ----  ----               -------
  Normal   Scheduled  45s   default-scheduler  Successfully assigned
                                               bad-pod to kind-worker
  Warning  Failed     30s   kubelet            Failed to pull image
                                               "nginx:invalid"
  Warning  Failed     30s   kubelet            Error: ErrImagePull
  Warning  BackOff    10s   kubelet            Back-off pulling image
                                               "nginx:invalid"
```

4. Understand the Error Messages

ErrImagePull

- Kubernetes cannot pull the container image from the registry.
- This mage name may be wrong, or the tag may not exist.
- It can also happen due to network or permission issues.

ImagePullBackOff

- Kubernetes repeatedly tried to pull the image but failed.
- It will keep retrying with an increasing delay (back-off).
- You will see this when the image does not exist or cannot be accessed.

5. Fix the Image and Verify Recovery

Use a valid image name and apply it.

```
kubectl set image pod/bad-pod bad-pod=nginx:stable
```

- This updates the Pod to use a valid NGINX image.
- Kubernetes will pull the image and start the container.
- Check the status again:

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
bad-pod	1/1	Running	0	2m

6. Image Pull Failure vs Application Crash

Image Pull Failure

- Container image cannot be downloaded.
- STATUS: ErrImagePull or ImagePullBackOff.
- Container never starts.
- Check image name, tag, registry access, and network.

Application Crash

- Image is pulled successfully.
- Container starts but application exits or crashes.
- STATUS: CrashLoopBackOff or Error.
- Check application logs with `kubectl logs <pod-name>`.
- Look for configuration errors or runtime issues inside the application.

7. Production Lesson

- Error messages are evidence. Read them carefully.
- Do not restart everything without understanding the cause.
- Check the actual error, investigate, fix the root issue, and then verify.
- This approach saves time and helps you solve real production problems.

9. Why Running a Pod Alone Is Not Enough

1. What Happens When a Standalone Pod Is Deleted?

- A standalone Pod is a single instance.
- If you delete it, it will be permanently removed from the cluster.
- Kubernetes does not automatically recreate it.
- You would have to manually create the Pod again.

```
# Create a Pod
kubectl run web-pod --image=nginx --port=80

# Delete the Pod
kubectl delete pod web-pod

# Check if it is gone
kubectl get pods
```

```
NAME      READY   STATUS    RESTARTS   AGE
No resources found in default namespace.
```

2. Why Production Workloads Need Controllers

- In production, applications must keep running all the time.
- Controllers automatically recreate Pods when they are deleted, fail, or become unhealthy.
- This ensures high availability and reduces manual work.
- Common controllers: Deployment, StatefulSet, DaemonSet, and Job.

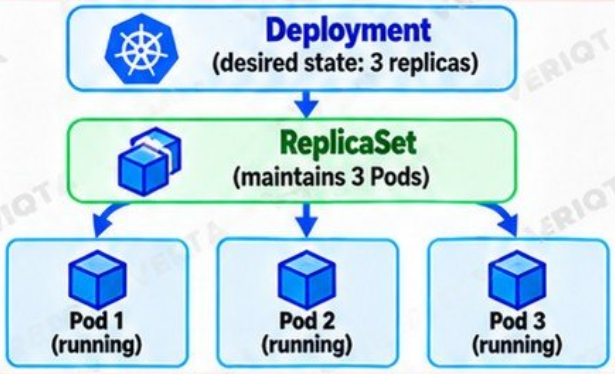
3. Desired State and Reconciliation

- You tell Kubernetes the desired state (e.g. 3 replicas of a Pod).
- Kubernetes continuously checks the actual state.
- If the actual state does not match the desired state, Kubernetes automatically takes action to fix it.
- This process is called reconciliation.

4. How a Deployment Manages ReplicaSets and Pods

- A Deployment is a controller that manages ReplicaSets.
- A ReplicaSet ensures the correct number of Pods are running.
- If a Pod is deleted or fails, the ReplicaSet creates a new one.
- The Deployment gives you higher-level features like updates, rollbacks, and scaling.

5. Architecture Diagram



6. Example: Deployment Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:stable
        ports:
        - containerPort: 80
```

7. What Happens If a Pod Is Deleted?

- With a standalone Pod: it is gone and not recreated.
- With a Deployment: a new Pod is automatically created to maintain the desired number of replicas.
- The ReplicaSet detects the missing Pod and creates a replacement.

8. Key Distinction

- A standalone Pod does not automatically get recreated after deletion.
- A Deployment (with ReplicaSet) continuously ensures the desired number of Pods are running.
- Use Deployments for production workloads, not standalone Pods.

9. Production Lesson

- Running a single Pod is useful for quick testing, but it is not reliable for production.
- Controllers like Deployment provide self-healing and high availability.
- Always use a controller for real applications.

10. Create Your First Kubernetes Deployment

1. Deployment YAML Example

This manifest creates a Deployment with 3 NGINX replicas.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  labels:
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:stable
        ports:
          - containerPort: 80
```



Deployment YAML

2. Understand the Key Sections

Section	What It Does
apiVersion	Specifies the Kubernetes API version (e.g. apps/v1).
kind	Type of resource (Deployment).
metadata	Name and labels for the Deployment.
spec.replicas	Desired number of Pod replicas (e.g. 3).
spec.selector	Tells the Deployment which Pods to manage using labels.
spec.template	Defines the Pod template, including labels and container specification.
containers	Container name, image, and ports for the application.

3. Apply the Deployment

Save the content to a file named deployment.yaml and run:

```
kubectl apply -f deployment.yaml
```

```
deployment.apps/web-deployment created
```

4. Verify the Resources

Check the Deployment, ReplicaSet, and Pods.

Check the Deployment:

```
kubectl get deployments
```

```
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
web-deployment      3/3     3            3           2m
```

Check the ReplicaSet:

```
kubectl get replicasets
```

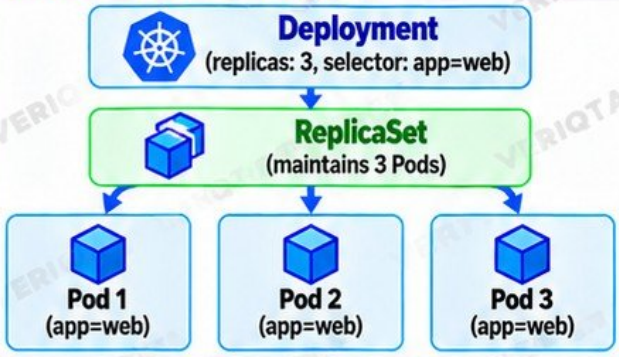
```
NAME                          DESIRED   CURRENT   READY   AGE
web-deployment-6f8d9c6b7      3         3         3       2m
```

Check the Pods:

```
kubectl get pods -l app=web
```

```
NAME                                READY   STATUS    RESTARTS   AGE
web-deployment-6f8d9c6b7-abc1       1/1     Running   0          2m
web-deployment-6f8d9c6b7-def2       1/1     Running   0          2m
web-deployment-6f8d9c6b7-ghi3       1/1     Running   0          2m
```

5. Architecture Diagram



6. Why Labels Must Match

- The Deployment uses a selector to find and manage Pods based on labels.
- The Pod template must have the same labels as the selector.
- If the labels do not match, the Deployment will not see or manage the Pods, and they will not be counted as replicas.

7. Common Failure: Labels Do Not Match

If the selector and template labels are different, the Deployment will not manage the Pods correctly.

```yaml
spec:
  selector:
    matchLabels:
      app: web    # selector is app=web
  template:
    metadata:
      labels:
        app: nginx  # label does not match
```

Result

- The Deployment may create new ReplicaSets repeatedly.
- Pods may not be counted as ready.
- You will see unexpected behavior and extra Pods.

8. Key Takeaways

- A Deployment manages ReplicaSets and ensures the desired number of Pods are always running.
- Pod labels must match the Deployment selector.
- Use kubectl to verify Deployments, ReplicaSets, and Pods.
- If things do not work, always check labels, events, and the actual state of the resources.

11. Self-Healing: Delete a Pod and Watch Kubernetes Respond

1. Create a Deployment with Two Replicas

Create an NGINX Deployment with 2 replicas.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  labels:
    app: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:stable
        ports:
          - containerPort: 80
```



Deployment (2 replicas)

2. Verify the Deployment and Pods

Check the Deployment and its managed Pods.

```
kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
web-deployment	2/2	2	2	1m

```
kubectl get pods -l app=web
```

NAME	READY	STATUS	RESTARTS	AGE
web-deployment-6d8f7c9b6-abc12	1/1	Running	0	1m
web-deployment-6d8f7c9b6-def34	1/1	Running	0	1m

3. Delete One Pod Deliberately

Delete one of the Pods managed by the Deployment.

```
kubectl delete pod web-deployment-6d8f7c9b6-abc12
```

```
pod "web-deployment-6d8f7c9b6-abc12" deleted
```

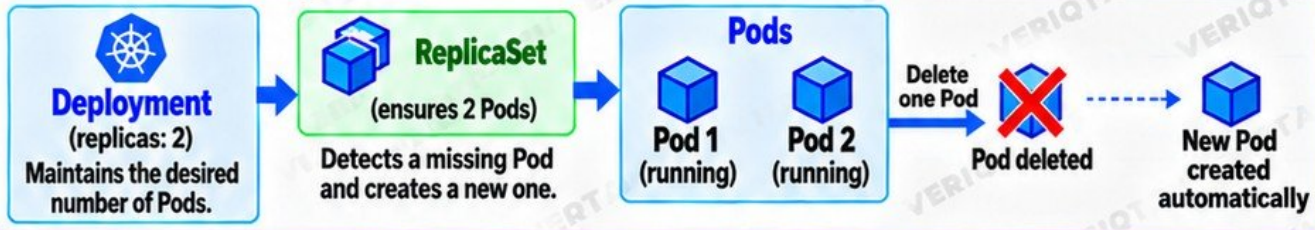
4. Watch Kubernetes Create a Replacement

Check the Pods again. A new Pod will be created automatically.

```
kubectl get pods -l app=web
```

NAME	READY	STATUS	RESTARTS	AGE
web-deployment-6d8f7c9b6-def34	1/1	Running	0	2m
web-deployment-6d8f7c9b6-ghi78	1/1	Running	0	10s

5. How Self-Healing Works



6. Understand Desired vs Current Replicas

Check the Deployment details to see desired and current replicas.

```
kubectl get deployment web-deployment
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
web-deployment	2/2	2	2	3m

```
kubectl describe deployment web-deployment | g rep -E "Replicas"
```

```
Replicas:       2 desired | 2 updated | 2 total | 2 available
```

7. Key Points to Remember

- The Deployment keeps the desired number of Pods running.
- If a Pod is deleted, the ReplicaSet creates a replacement.
- The new Pod may have a different name.
- This process is automatic and part of Kubernetes self-healing.
- You did not need to manually create the Pod again.

8. Production Insight

- Kubernetes restores the declared number of Pods, but it cannot guarantee that a failing application becomes healthy.
- If the application has a bug, the new Pod will also fail.
- You still need monitoring, health checks, and proper application design.

9. Common Failure Scenarios

- If the Deployment has 1 replica, there is no extra Pod.
- If the node is unhealthy, a replacement may be delayed.
- Image pull errors or application crashes will create new Pods, but they may also fail.
- Always check events and logs if the new Pod is not running.

12. Scaling Your Application from One Pod to Three

1. Understand Horizontal Scaling

- Horizontal scaling means running more replicas of your application.
- It increases the number of Pods to handle more traffic and improve availability.
- In Kubernetes, you scale a Deployment by changing the desired number of replicas.
- Each replica runs in a separate Pod.

2. Current State: One Replica

Check the current number of replicas.

```
kubectl get deployment web
```

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    1/1     1            1           2m
```

3. Scale to Three Replicas

Increase the number of replicas to 3.

```
kubectl scale deployment web --replicas=3
```

```
deployment.apps/web scaled
```

4. Verify the Scaling Result

Check the Deployment and Pods again.

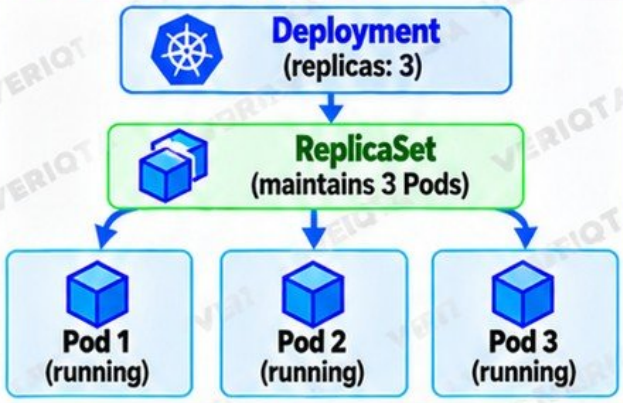
```
kubectl get deployment web
```

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    3/3     3            3           3m
```

```
kubectl get pods -l app=web
```

```
NAME                   READY   STATUS    RESTARTS   AGE
web-6d8f7c9b6-abc12    1/1     Running   0          2m
web-6d8f7c9b6-def34    1/1     Running   0          2m
web-6d8f7c9b6-ghi78    1/1     Running   0          2m
```

5. How Scaling Works



6. What Scaling Changes

- Increases the number of Pod replicas.
- The ReplicaSet creates additional Pods.
- Helps handle more traffic and improves availability.

7. What Scaling Does Not Change

- It does not automatically provide external access.
- It does not automatically create a Service.
- It does not change your application code.
- It does not guarantee that your application can handle more traffic without proper design.

8. Why Multiple Replicas Do Not Provide External Access

- Pods have internal IP addresses and are reachable only inside the cluster.
- To access your application from outside, you need a Service (e.g. ClusterIP, NodePort, or LoadBalancer).
- Multiple replicas improve availability, but a Service is required to expose them.

9. Production Awareness

- More replicas require sufficient cluster resources (CPU, memory, and node capacity).
- Your application must be designed to handle multiple instances (stateless or external shared state).
- Check resource usage and cluster limits when scaling.
- Scaling helps with availability and performance, but it is not a complete solution for all problems.

13. Update Your Application Without Recreating Everything Manually

1. Understand Rolling Updates

- A rolling update replaces Pods gradually with new ones.
- It keeps your application available during the update.
- Kubernetes creates new Pods with the new version and then removes the old Pods.
- Each update creates a new Deployment revision.

2. Check the Current Deployment

See the current image and revision.

```
kubectl get deployment web
```

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    3/3     3            3           10m
```

View more details:

```
kubectl describe deployment web | grep -i image
```

```
Image:        nginx:1.21
```

3. Update the Application Image

Change the image to a new version (e.g. nginx:1.22).

```
kubectl set image deployment/web \
  nginx=nginx:1.22
```

```
deployment.apps/web image updated
```

4. Observe the Rollout Progress

Watch the rolling update in real time.

```
kubectl rollout status deployment/web
```

```
Waiting for deployment "web" rollout to finish...
deployment "web" successfully rolled out
```

5. How a Rolling Update Works

Current Version (nginx:1.21)



3 running Pods

Rolling Update (in progress)

- Create new Pods with new version.
- Gradually route traffic to new Pods.
- Remove old Pods.

New Version (nginx:1.22)



3 running Pods

Result

- Application stays available.
- Pods are replaced gradually.
- No manual Pod deletion required.
- A new Deployment revision is created.

6. Check Rollout History

See previous revisions and the images used.

```
kubectl rollout history deployment/web
```

```
REVISION        CHANGE-CAUSE
1               <none>
2               <none>
```

7. Roll Back to a Previous Version

If the new version has issues, roll back.

```
kubectl rollout undo deployment/web
```

```
deployment.apps/web rolled back
```

Verify the rollback:

```
kubectl get deployment web
```

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    3/3     3            3           15m
```

8. Common Failure: New Image Fails to Start

- The new Pods get created but fail to start.
- Rollout may get stuck or roll back automatically.
- Check Pod events and logs to find the real cause.

Check the status: `kubectl get pods -l app=web`

9. Production Lesson



- A successful rollout only means the new Pods are running.
- It does not guarantee that the application is working correctly.
- Always verify application health, logs, and functionality after an update.

10. Key Takeaways

- Use rolling updates for zero-downtime deployments.
- Each update creates a new Deployment revision.
- You can inspect history and roll back if needed.
- A rollout success does not replace application-level testing.
- Always monitor your application after an update.

14. Why Pods Need Services: The Problem with Changing IP Addresses

1. Pods Can Receive Different IP Addresses

- Each Pod gets a unique IP address inside the cluster.
- When a Pod is deleted and recreated, it can get a different IP address.
- Pod IP addresses are temporary and not stable.
- You should not hardcode or depend on individual Pod IPs in your applications.

2. Example: Pod IP Address Changes

Check the Pod IP addresses.

```
kubectl get pods -o wide -l app=web
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
web-6d8f7c9b6-abc12	1/1	Running	0	2m	10.244.1.12	node1
web-6d8f7c9b6-def34	1/1	Running	0	2m	10.244.2.15	node1

- If a Pod is deleted and recreated, the new Pod will have a different IP address (e.g. 10.244.3.8).

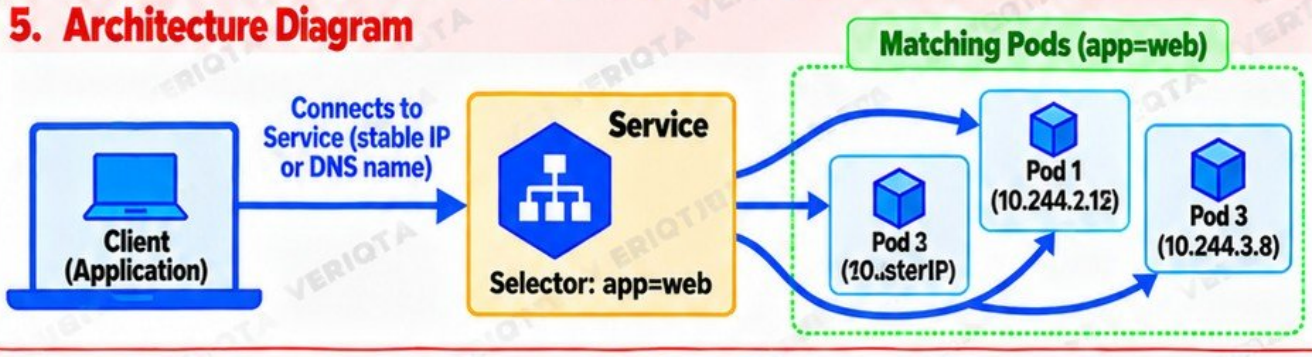
3. Why a Service Is Needed

- A Service provides a stable, consistent access point for a set of Pods.
- Pods can be created, deleted, or replaced, but the Service keeps the same virtual IP (ClusterIP) and DNS name.
- Applications connect to the Service, not to individual Pod IP addresses.
- The Service automatically routes traffic to healthy, matching Pods.

4. How a Service Works

- A Service selects Pods using labels.
- It keeps a stable virtual IP address (ClusterIP).
- It load balances traffic to all matching Pods.
- If Pods are replaced, the Service continues to work without changes.
- The Service only routes traffic to Pods that match its selector labels.

5. Architecture Diagram



6. Service and Pod Labels Must Match

The Service uses a selector to find matching Pods.

Service selector:

```
spec:
  selector:
    app: web
```

Pod labels:

```
metadata:
  labels:
    app: web
```

- The labels in the Service selector must match the labels on the Pods, otherwise the Service will not route traffic to them.

7. Example: Create a ClusterIP Service

Create a Service to expose the Deployment internally.

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80
```

Apply the Service:

```
kubectl apply -f service.yaml
```

Check the Service:

```
kubectl get svc
```

8. Common Beginner Mistake

Thinking a Service creates or manages Pods.

- A Service does not create, delete, or manage Pods.
- A Service only provides a stable access point and routes traffic to existing matching Pods.

9. Key Takeaways

- Pod IP addresses are temporary and can change.
- Use a Service for a stable access point.
- Service selectors must match Pod labels.
- A Service does not create or manage Pods.
- Always connect to the Service, not individual Pod IPs.

15. Create Your First ClusterIP Service

1. ClusterIP Service YAML Example

Create a Service to expose the existing NGINX Deployment inside the cluster.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - name: http
      port: 80
      targetPort: 80
```



2. Understand the Key Fields

Field	What It Does
apiVersion	Specifies the Kubernetes API version (e.g. v1).
kind	Type of resource (Service).
metadata	Name of the Service.
spec.type	ClusterIP makes the Service accessible only inside the cluster.
spec.selector	Selects the Pods to send traffic to (must match Pod labels).
spec.ports.port	Port that the Service exposes.
spec.ports.targetPort	Port on the Pods to receive traffic (e.g. NGINX port 80).

3. Apply the Service

Save the YAML to a file named service.yaml and apply it.

```
kubectl apply -f service.yaml
```

```
service/web-service created
```

4. Verify the Service

Check the Service details.

```
kubectl get svc
```

```
NAME          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
web-service   ClusterIP   10.96.214.55   <none>        80/TCP    2m
```

5. Check the Endpoints

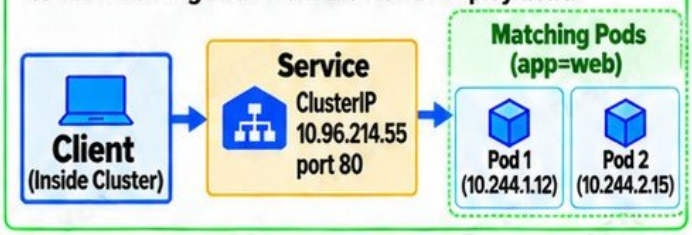
View the Pods backing the Service.

```
kubectl get endpointslices
```

```
NAME                ADDRESSES                 PORTS   AGE
web-service-abc12   10.244.1.12,10.244.2.15   80      2m
```

6. How It Connects to the Deployment

The Service uses the selector to find and route traffic to the matching Pods from the NGINX Deployment.



7. Service DNS Name

You can access the Service using its DNS name from inside the cluster.

Format:

```
<service-name>.<namespace>.svc.cluster.local
```

Example (default namespace):

```
http://web-service.default.svc.cluster.local:80
```

- This DNS name always points to the ClusterIP and routes traffic to the matching Pods.

8. Test Access from Another Pod

Run a temporary Pod and access the Service.

```
kubectl run test-pod --image=busybox -it --rm -- sh
# inside the pod
wget -qO- http://web-service.default.svc.cluster.local
```

```
Connecting to web-service.default.svc.cluster.local (10.96.214.55:80)
<!DOCTYPE html>
<html>
<head><title>Welcome to nginx!</title></head>
...
```

9. Common Failure: No Ready Backends

If the Service selector does not match any Pod labels, the Service will have no endpoints and cannot route traffic.

- Selector labels do not match Pod labels.
- No endpoints are created.
- The Service has a ClusterIP but no ready backends.
- Traffic to the Service will fail.

10. Key Takeaways

- ClusterIP is accessible only inside the cluster.
- The Service uses a selector to find matching Pods.
- port is the Service port, targetPort is the Pod port.
- The Service provides a stable virtual IP and DNS name.
- A Service does not create or manage Pods.
- Always check endpoints if the Service is not working.

16. Access Your Application from Your Laptop

1. Internal Cluster Networking vs Local Access

- ClusterIP Services are only accessible inside the cluster.
- Your laptop is outside the cluster, so you cannot directly access a ClusterIP.
- `kubectl port-forward` creates a temporary secure tunnel from your laptop to the Service.
- This allows you to access the application running inside the cluster.

2. Use kubectl port-forward

Forward local port 8080 to the Service port 80.

```
kubectl port-forward service/web-service 8080:80
```

You should see output like:

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

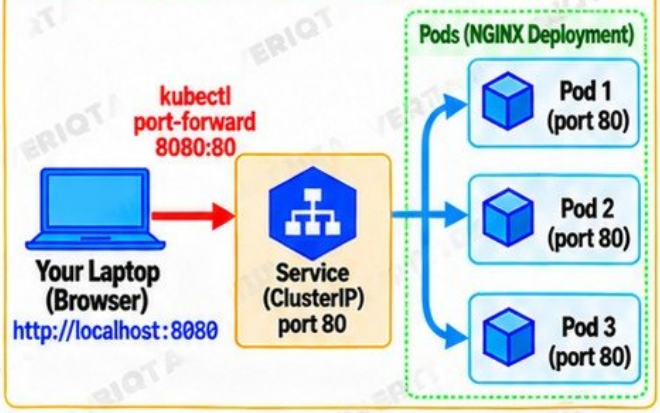
3. Open the Application in Your Browser

After port forwarding is running, open this URL in your browser:

`http://localhost:8080`



4. Connection Flow Diagram



5. Compare Kubernetes Service Types (Beginner Level)

Service Type	What It Does	How to Access	Use Case
ClusterIP	Internal IP accessible only inside the cluster.	Only from inside the cluster (or using port-forward).	Default type. Used for internal communication between services.
NodePort	Exposes the Service on a port on each cluster node.	`http://<NodeIP>:<NodePort>`	Simple way to access from outside the cluster (not for production).
LoadBalancer	Creates an external load balancer (with a public IP, if supported).	`http://<External-IP>`	Used in cloud environments for external access (production).

6. Trace the Connection

1. You access `http://localhost:8080` in your browser.
2. The request goes to `kubectl port-forward`.
3. kubectl forwards the request to the Service (ClusterIP).
4. The Service routes the request to one of the matching Pods.
5. The NGINX container responds and the page is displayed in your browser.

7. Security Reminder

- Port forwarding is a temporary access method for development and testing.
- It is not a production exposure strategy.
- Do not use port forwarding to expose applications to the internet.
- For real external access, use NodePort or LoadBalancer with proper security, authentication, and network controls.

8. Common Beginner Mistakes

- Trying to access a ClusterIP directly from your laptop.
- Using the wrong service name or port.
- Forgetting to keep the port-forward command running.
- Expecting port forwarding to provide a public URL.

9. Key Takeaways

- ClusterIP is internal to the cluster.
- Use kubectl port-forward to access it from your laptop.
- You can also use NodePort or LoadBalancer for external access.
- Port forwarding is temporary and not for production.
- Always secure and monitor external access methods.

17. Troubleshoot a Service That Cannot Reach Its Pods

1. Identify the Problem

The Service exists, but it has no matching backends.

```
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
web-service	ClusterIP	10.96.214.55	<none>	80/TCP	5m

```
kubectl get endpointslices
```

NAME	ADDRESSES	PORTS	AGE
web-service-abc12	<none>	80	5m

✗ The Service has no endpoints, so it cannot reach any Pods.

2. Compare Service Selectors with Pod Labels

The Service selector must match the Pod labels.

Service Selector

```
spec:
  selector:
    app: web
```

Pod Labels

```
metadata:
  labels:
    app: nginx
```

✗ These labels do not match, so the Service will not find any Pods.

✓ Update the Service selector to match the Pod labels (e.g. app: nginx).

3. Check Pod Readiness

Make sure the Pods are running and ready.

```
kubectl get pods -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-6d8f7c9b6-abc12	1/1	Running	0	3m
web-6d8f7c9b6-def34	1/1	Running	0	3m

✓ Pods are running and ready.

4. Check Port and targetPort

The Service port and targetPort must match the container port.

Service Ports

```
spec:
  ports:
  - port: 80
    targetPort: 80
```

Pod Container Port

```
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

✓ The targetPort (80) matches the container port (80).

5. Inspect EndpointSlices in Detail

EndpointSlices show the actual backends for the Service.

```
kubectl get endpointslices -l kubernetes.io/service-name=web-service
```

NAME	ADDRESSES	PORTS	AGE
web-service-abc12	10.244.1.12,10.244.2.15	80	5m

✓ Endpoints now show the IP addresses of the matching Pods.

6. Correct the Service and Verify

Update the Service selector (if needed) and test again.

```
kubectl patch service web-service \
  -p '{"spec":{"selector":{"app":"nginx"}}}'
```

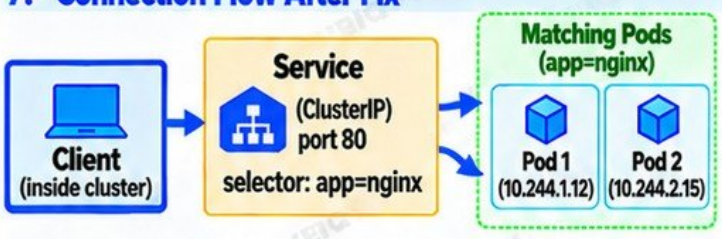
Check the endpoints again:

```
kubectl get endpointslices
```

NAME	ADDRESSES	PORTS	AGE
web-service-abc12	10.244.1.12,10.244.2.15	80	6m

✓ The Service now routes traffic to the Pods.

7. Connection Flow After Fix



8. Production Insight

- A Service can exist and still have no usable application endpoints.
- Always verify selectors, Pod labels, and Pod readiness when troubleshooting.
- EndpointSlices are the source of truth for backends.
- Check port and targetPort if the application is not reachable.
- Do not assume a Service automatically creates or manages Pods.

9. Common Failure Reasons

- Service selector does not match Pod labels.
- Pods are not ready.
- Wrong port or targetPort.
- Application is crashing inside the container.
- Expecting the Service to create or manage Pods.

10. Key Takeaways

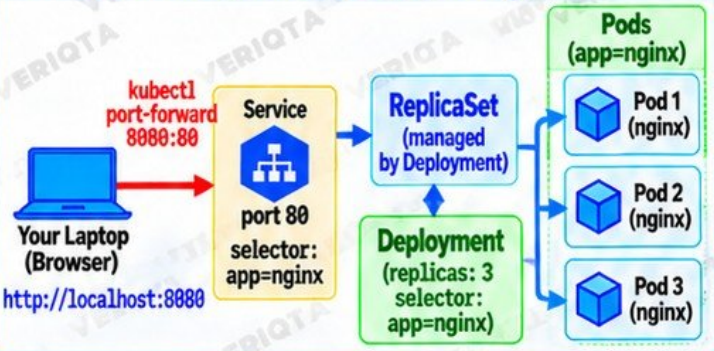
- The Service selector must match the Pod labels.
- Use EndpointSlices to confirm backends.
- Ensure Pods are running and ready.
- Verify port and targetPort configuration.
- A Service can exist without endpoints.
- Always check the full connection flow when troubleshooting.

18. Your First Complete Kubernetes Application Architecture

1. Complete Architecture Overview

- A Deployment creates and manages a ReplicaSet.
- The ReplicaSet creates and maintains the Pods.
- A Service provides a stable access point to the Pods.
- Labels and selectors connect the resources.
- When a Pod is replaced, the Service continues to route traffic to the new Pod automatically.

2. Application Architecture Diagram



3. Trace a Request from Browser to Pod

Follow what happens when you open the application.

1. You run: `kubectl port-forward service/web 8080:80`
2. You open `http://localhost:8080` in your browser.
3. The request goes to the Service (ClusterIP).
4. The Service selects one of the matching Pods (app=nginx).
5. The request is forwarded to the NGINX container.
6. The NGINX container returns the web page.

4. Key Resource Relationships

Resource	What It Does	Key Configuration
Deployment	Manages ReplicaSets and ensures desired number of Pods.	replicas, selector, template
ReplicaSet	Creates and maintains the Pods.	selector (matches Pod labels)
Pod	Runs the container (e.g. NGINX).	labels (e.g. app=nginx)
Service	Provides a stable access point and routes traffic to matching Pods.	type, selector, port, targetPort

5. Labels and Selectors Connection

The Service selector must match the Pod labels (from the Deployment).

Deployment Pod Labels (in template)

```
metadata:
  labels:
    app: nginx
```

=

Service Selector

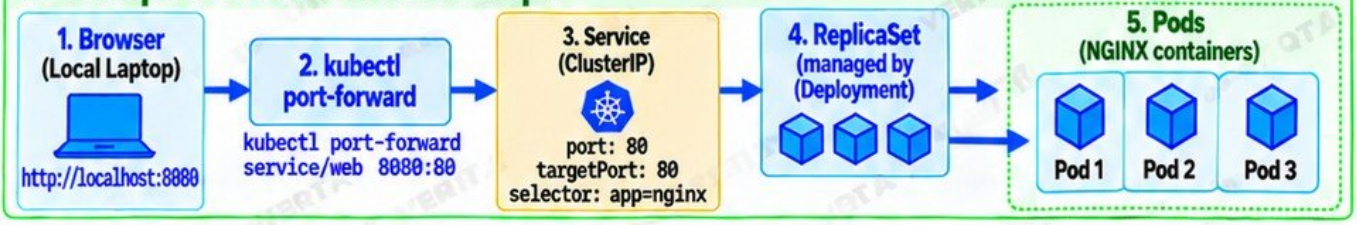
```
spec:
  selector:
    app: nginx
```

✓ The labels and selector must match for the Service to find the Pods.

6. What Happens When a Pod Is Replaced?

- If a Pod is deleted or fails, the ReplicaSet creates a new Pod.
- The new Pod gets a new IP address.
- The new Pod has the same labels (app=nginx).
- The Service automatically routes traffic to the new Pod.
- Your application remains available without manual changes.

7. Complete Flow with Real Example



8. Practice: Which Resource Does What?

Match each responsibility to the correct Kubernetes resource.

1. Creates and manages ReplicaSets.
2. Creates and maintains Pods.
3. Runs the application container (e.g. NGINX).
4. Provides a stable access point and routes traffic.

Options:
A. Deployment
B. ReplicaSet
C. Pod
D. Service

9. Key Takeaways

- A Deployment manages ReplicaSets and Pods.
- A Service routes traffic to matching Pods using labels.
- Labels and selectors connect the resources.
- A Pod replacement does not break the application.
- The Service provides a stable access point, even when Pods are recreated.
- Understand the role of each resource in the application.

19. Your First Kubernetes Production Readiness Checklist

1. Running Pods Are Not Enough

- Pods can be Running, but the application inside may still be failing.
- Kubernetes only checks if the container is alive, not if your application is working correctly.
- You need probes, proper configuration, monitoring, and testing to know if the application is truly healthy.
- Always test the actual application, not just the Pod status.

2. Readiness and Liveness Probes

Liveness Probe

- Checks if the application is still running.
- If it fails, Kubernetes restarts the container.
- Use for application health (e.g. /health or /).

Readiness Probe

- Checks if the application is ready to receive traffic.
- If it fails, the Pod is removed from Service endpoints.
- Use for startup checks (e.g. /ready).

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 80
  initialDelaySeconds: 10
  periodSeconds: 10
```

```yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 5
```

3. CPU and Memory Requests and Limits

Why Use Them?

- Requests guarantee the minimum resources a Pod needs.
- Limits set the maximum resources a Pod can use.
- They help with stability, fair resource sharing, and preventing resource exhaustion.

Example

```yaml
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

4. Use Versioned Container Images

- Always use a specific version tag (e.g. `nginx:1.25`) instead of latest.
- Versioned images make deployments predictable and reproducible.
- They help you avoid unexpected changes if the image is updated.
- They make rollbacks easier when something goes wrong.

Good

`nginx:1.25`

Not Recommended

`nginx:latest`

5. Basic Container Security

- Use non-root users when possible.
- Avoid running containers with unnecessary privileges.
- Do not use `privileged: true` unless required.
- Drop unnecessary Linux capabilities.
- Use trusted and minimal base images (e.g. `nginx`, `alpine`).

```yaml
securityContext:
  runAsNonRoot: true
  runAsUser: 101
  allowPrivilegeEscalation: false
  capabilitios:
    drop: ["ALL"]
```

6. When to Inspect and What to Check

Logs

Check application logs for errors.

```
kubectl logs <pod-name>
```

Events

Check events for Pod and Deployment issues.

```
kubectl get events --sort-by=.lastTimestamp
```

Resource Usage

Check CPU and memory usage.

```
kubectl top pods
kubectl top nodes
```

Rollout Status

Check deployment rollout progress.

```
kubectl rollout status deployment <name>
```

7. Key Checks Before Considering It Ready

- Pods are running and ready.
- Readiness and liveness probes are configured.
- CPU and memory requests and limits are set.
- Using a versioned container image.
- Container security is configured.
- No crash loops in logs.
- No critical errors in events.
- Rollout completed successfully.
- Application is accessible and working.

8. Junior Engineer Insight

- A successful Deployment only means Kubernetes created the Pods.
- Running Pods only means the containers are alive.
- Application health means the application inside the container is working correctly.
- User availability means real users can access and use the application.
- You must check all four, they are different things.

9. Key Takeaways

- Running Pods are not enough to prove health.
- Use readiness and liveness probes.
- Set CPU and memory requests and limits.
- Use versioned images.
- Follow container security best practices.
- Inspect logs, events, resource usage, and rollout status.
- Always test the actual application.

20. Final Hands-On Challenge: Deploy, Break, Fix, and Verify

1. Deploy an NGINX Application

Create a Deployment manifest for NGINX.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

Apply the manifest.

```
kubectl apply -f deployment.yaml
```

2. Run Two Replicas and Create a ClusterIP Service

Create a ClusterIP Service manifest.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
```

Apply the manifest.

```
kubectl apply -f service.yaml
```

3. Access the Application Using Port Forwarding

Forward local port 8080 to the Service.

```
kubectl port-forward service/web-service 8080:80
```

Open in your browser.

```
http://localhost:8080
```



4. Delete a Managed Pod and Verify Replacement

List the Pods.

```
kubectl get pods -l app=nginx
```

Delete one Pod.

```
kubectl delete pod <pod-name>
```

Watch Kubernetes create a new Pod.

```
kubectl get pods -l app=nginx -w
```

5. Break It: Use a Wrong Service Selector

Edit the Service and change the selector to a wrong label (e.g. app: web).

```yaml
spec:
  selector:
    app: web   # wrong label
```

Apply the change.

```
kubectl apply -f service.yaml
```

Check endpoints (should be empty).

```
kubectl get endpointslices
```

6. Diagnose the Failure

Check the Service and endpoints.

```
kubectl get svc
kubectl get endpointslices
```

NAME	ADDRESSES	PORTS	AGE
web-service	<none>	80	2m

The Service has no matching backends, so it cannot reach any Pods.

7. Fix It: Restore the Correct Selector

Update the Service selector back to app: nginx.

```yaml
spec:
  selector:
    app: nginx   # correct label
```

Apply the change and verify.

```
kubectl apply -f service.yaml
kubectl get endpointslices
```

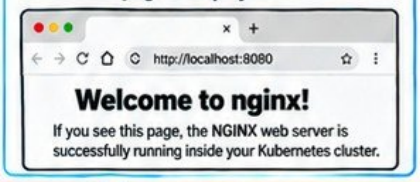
Endpoints now show the IP addresses of the matching Pods.

8. Verify Application Access

Test again using port forwarding (if not already running).

```
kubectl port-forward service/web-service 8080:80
```

Open http://localhost:8080 and confirm the NGINX page is displayed.



9. Clean Up the Lab Resources

Delete the Service and Deployment.

```
kubectl delete service web-service
kubectl delete deployment web
```

Verify deletion.

```
kubectl get pods
kubectl get svc
```

All resources are removed.

10. Final Knowledge Check

Explain the role of each resource in your own words.

1. What does a Pod do?
2. What does a Deployment do?
3. What does a ReplicaSet do?
4. What does a Service do?
5. How do they work together to run a real application?

11. Key Takeaways

- A Deployment creates and manages ReplicaSets and Pods.
- A ReplicaSet maintains the desired number of Pods.
- A Pod runs the application container.
- A Service provides a stable access point to the Pods.
- Kubernetes replaces deleted or failed Pods automatically.
- A Service can exist without backends if the selector is wrong.
- Always verify the actual application, not just Pod status.
- Clean up resources after testing.